SOFTWARE ENGINEERING LAB | LAB 28

Timeouts & Deadlines

Mencegah Satu Dependency Lambat Menghancurkan Seluruh Ekosistem Distributed System

Lukman / lukman-ss • github.com/lukman-ss/software-engineering-lab

Slow Dependency Lebih Berbahaya daripada Dead Dependency

Fast failure melepaskan resource seketika; latency tinggi menahan thread pool

FAST FAILURE (DEAD)

- Mengembalikan ECONNREFUSED atau reset paket secara instan
- Worker thread langsung dilepas ke pool dalam orde mikrodetik
- Sistem caller tetap responsif untuk request lain

SLOW DEPENDENCY

- Latency melonjak drastis, misal dari 300ms menjadi 60 detik
- Request in-flight tertahan di thread pool, memory, dan socket
- Endpoint independen (/profile, /health) kehabisan worker thread

FORMULA LITTLE'S LAW

- L = λ × W (Konkurensi = Arrival Rate × Latency)
- Jika W naik 200× (0.3s ke 60s), kapasitas L meledak 200× lipat
- Thread pool 100 worker hanya mampu melayani 1.6 QPS pada W=60s

Fast failure frees resources immediately; slow dependencies hoard threads until total starvation.

Timeout Adalah Resource Guardrail, Bukan Tebakan Aman

Empat prinsip fundamental dalam perancangan time budget

TIMEOUT ≠ FAILURE

- Timeout berarti caller tidak menerima response tepat waktu
- Status backend adalah indeterminate: mungkin sukses, mungkin gagal
- Jangan pernah mengasumsikan transaksi dibatalkan hanya karena timeout

BUDGET HIERARKIS

- Deadline harus diwariskan dari hulu ke hilir distributed trace
- Child timeout tidak boleh melebihi sisa waktu parent deadline
- Downstream harus membatalkan eksekusi jika parent telah timeout

PROTEKSI TOTAL

- Retry tanpa jitter menciptakan thundering herd retry storm
- HTTP timeout tidak melindungi DB connection pool dari query liar
- Gunakan statement_timeout pada database storage layer

Timeout means caller gave up waiting, not that the backend operation was aborted.

Komponen Pertahanan Terdistribusi Lab 28

Empat subsistem mandiri yang bekerja bersinergi menahan kegagalan

1. CONTEXT DEADLINE

- Package internal/deadline
- Pewarisan context.WithTimeout dengan channel non-blocking
- Pencegahan goroutine leak saat caller membatalkan eksekusi

2. RETRY WITH FULL JITTER

- Package internal/retry
- Exponential backoff 2^(attempt-1) dikombinasikan seragam acak
- Menghilangkan thundering herd antar worker

3. CIRCUIT BREAKER & DEDUP

- Package internal/circuit: State CLOSED, OPEN, HALF_OPEN
- Package internal/idempotency: Store in-memory dengan lazy eviction
- Melindungi downstream yang kolaps dan mencegah double charge

A resilient pipeline combines deadlines, jittered retries, circuit breaking, and idempotency.

Implementasi Context Deadline di Go

Memastikan child budget tunduk pada parent context tanpa memory leak

CHILD TIMEOUT BUDGET

- childCtx, cancel := context.WithTimeout(ctx, budget)
- defer cancel() selalu dipanggil untuk membersihkan timer
- Child deadline = min(parent.Deadline(), now + budget)

BUFFERED CHANNEL (CAP=1)

- done := make(chan error, 1)
- Kapasitas 1 menjamin goroutine pekerja tidak pernah block
- Mencegah goroutine leak saat select memilih childCtx.Done()

BUKTI PENGUJIAN

- TestExecuteWithBudget_Timeout: task 100ms dipotong budget 20ms
- TestExecuteWithBudget_ParentTimeoutInherited: parent 20ms batalkan budget 500ms
- Mengembalikan context.DeadlineExceeded secara deterministik

Always buffer the completion channel to capacity 1 to prevent goroutine leaks on timeout.

Pewarisan Deadline dalam Hierarki Distributed Trace

Waktu tidak bertambah di downstream; deadline selalu menyusut

SKENARIO API GATEWAY

- API Gateway menetapkan timeout 500ms untuk request user
- Service A butuh 150ms pemrosesan lokal sebelum memanggil Service B
- Budget aktual yang diteruskan ke Service B tersisa 350ms

DOWNSTREAM OVER-ESTIMATION

- Jika Service B membuat timeout 2 detik sendiri tanpa parent ctx
- Saat detik ke-1 user sudah menerima 504 Gateway Timeout
- Service B tetap membakar CPU untuk request yang tidak lagi dinanti

MEKANISME GOLANG

- context.WithTimeout secara otomatis mewarisi parent deadline
- Jika parent deadline lebih pendek dari budget baru, parent menang
- Sub-operasi berhenti bersamaan saat parent deadline terpicu

Time budgets strictly shrink as calls descend through microservice trees.

Bahaya Retry Tanpa Jitter: Thundering Herd

Retry deterministik menyinkronkan ribuan client menjadi gelombang perusak

SINKRONISASI CLIENT

- 1.000 client gagal bersamaan saat database spike sejenak
- Semua client menjadwalkan retry tepat pada attempt × 100ms
- Downstream dihantam gelombang request periodik yang sinkron

RETRY STORM COLLAPSE

- Server downstream yang baru pulih langsung tumbang kembali
- Cascading failure merambat naik ke API Gateway
- Load amplification: 1 request asli menjadi 3-5 request bersamaan

SOLUSI AWS ARCHITECTURE

- Bukan sekadar exponential backoff berlipat ganda
- Wajib menambahkan randomisasi penuh (Full Jitter)
- Meredistribusikan beban secara merata di sepanjang sumbu waktu

Deterministic retries synchronize clients into destructive thundering herds.

Matematika Exponential Backoff dengan Full Jitter

sleep = random(0, min(maxBackoff, baseBackoff × 2^(attempt-1)))

FORMULA KALKULASI

- multiplier := 1 << uint(attempt - 1)
- temp := min(MaxBackoff, BaseBackoff * multiplier)
- sleep := rand.Float64() * temp (Distribusi Seragam)

KEUNGGULAN FULL JITTER

- AWS Research membuktikan Full Jitter meminimalkan total waktu kerja
- Mengurangi server contention hingga titik terendah dibanding Equal Jitter
- Setiap client memiliki waktu tunggu yang independen secara statistik

BATASAN IMPLEMENTASI

- attempt <= 0 ditangani aman dengan membatasi shift bit
- Backoff dibatasi MaxBackoff untuk mencegah overflow
- Default zero config: 3 attempts, base 10ms, max 100ms

Full jitter samples uniformly from zero to the exponential ceiling, maximizing dispersion.

Loop Eksekusi Retrier dengan Respek Context

Retry loop yang berhenti seketika saat context timeout tercapai

STRUKTUR EKSEKUSI

- Iterasi 1 hingga cfg.MaxAttempts secara berurutan
- Mengeksekusi operasi fn(ctx) di tiap iterasi
- Jika err == nil, fungsi langsung return tanpa sleep

RESPEK TERHADAP CONTEXT

- Sleep dilakukan menggunakan select { case <-time.After(): ... }
- case <-ctx.Done(): return ctx.Err() membatalkan jeda tidur
- Tidak membuang waktu menunggu jika caller sudah membatalkan

ERROR HANDLING

- Mengembalikan ErrMaxRetriesExceeded jika semua percobaan gagal
- Membungkus lastErr untuk debugging riwayat kegagalan
- Test membuktikan retry berhenti saat parent context hangus

A retrier must abort its sleep immediately when the governing context is cancelled.

Circuit Breaker: Menghindari Hammering Downstream

Tiga status proteksi: CLOSED, OPEN, dan HALF-OPEN

CLOSED STATE (NORMAL)

- Semua request diteruskan langsung ke downstream service
- Menghitung kegagalan berturut-turut (consecutive failures)
- Jika failure count >= FailureThreshold (misal 2), transisi ke OPEN

OPEN STATE (ISOLASI)

- Semua request langsung ditolak dengan ErrCircuitOpen
- Tidak ada satupun request yang dikirim ke downstream
- Memberikan ruang napas bagi downstream yang kolaps

HALF-OPEN STATE (UJI COBA)

- Aktif otomatis setelah CooldownWindow (misal 50ms) berlalu
- Mengizinkan request uji coba masuk secara bertahap
- Sukses berturut-turut memulihkan ke CLOSED; 1 gagal kembali ke OPEN

Circuit breaker fails fast to give dying downstream dependencies room to recover.

Thread-Safe State Transitions pada Breaker

Sinkronisasi state menggunakan sync.RWMutex untuk throughput tinggi

METODE ALLOW() & READ LOCK

- b.mu.RLock() membaca state saat ini secara konkuren
- Mengecek apakah durasi OPEN telah melampaui CooldownWindow
- Promosi ke b.mu.Lock() hanya saat transisi status ke HALF_OPEN

RECORD SUCCESS & FAILURE

- b.mu.Lock() eksklusif memperbarui counter dan timestamp
- Di HALF_OPEN: 1 kegagalan langsung memicu OPEN kembali
- Di HALF_OPEN: sukses mencapai threshold memicu pemulihan ke CLOSED

VERIFIKASI INTEGRITAS

- TestCircuitBreaker_StateTransitions memvalidasi siklus penuh
- TestCircuitBreaker_HalfOpenFailureTripsOpen memvalidasi sensitivitas
- Lolos pengujian race detector tanpa race condition

Fast reads under RLock; atomic state mutations under exclusive Lock.

Ambiguitas Status Saat Operasi Mengalami Timeout

Apakah request benar-benar gagal atau berhasil tapi respon hilang?

SKENARIO JARINGAN ASINKRON

- Client mengirim request POST /charges untuk potong saldo $100
- Database bank sukses mendebit saldo nasabah di backend
- Kabel fiber putus atau reverse proxy timeout sebelum ACK terkirim

DILEMA CLIENT CALLER

- Client hanya menerima pesan: context deadline exceeded
- Client TIDAK BISA membedakan apakah transaksi sukses atau gagal
- Jika caller langsung melakukan retry mentah, saldo terpotong $200

PERLINDUNGAN IDEMPOTENSI

- Setiap operasi yang mengubah state wajib menyertakan Idempotency-Key
- Server menyimpan hash request dan response hasil pemrosesan
- Retry dengan key yang sama tidak boleh mengeksekusi ulang transaksi

Timeout is indeterminate: the database write may have committed even if the network dropped.

In-Memory Idempotency Store & Lazy Eviction

Penyimpanan response dengan TTL dan pembersihan pasif tanpa timer leak

RECORD STRUKTUR & TTL

- type Record struct { Response string; CreatedAt time.Time }
- Set(key, response) menyimpan payload dengan timestamp saat ini
- Store menjaga masa retensi data sesuai configured TTL (misal 50ms)

LAZY EVICTION PATTERN

- Tidak menggunakan background goroutine periodik pemakan resource
- Get(key) memeriksa selisih time.Since(rec.CreatedAt) > s.ttl
- Record kadaluarsa dihapus seketika saat diakses pembaca

WRITE LOCK PADA METHOD GET

- Method Get() menggunakan s.mu.Lock(), bukan RLock()
- Penting karena lazy eviction melakukan mutasi delete(s.records, key)
- Menjamin konsistensi memori dan lolos race detector di bawah 50 goroutine

Lazy eviction cleans expired keys during lookups, eliminating background cleaner goroutines.

Integrasi: Retry Loop Bersanding dengan Circuit Breaker

Breaker memotong rantai retry seketika saat downstream terbukti down

EKSEKUSI BERSAMA

- Retrier dikonfigurasi 4 attempt dengan backoff 5ms - 20ms
- Circuit Breaker memiliki failure threshold 2 dan cooldown 50ms
- Downstream disimulasikan selalu mengalami kegagalan 500 error

PEMOTONGAN CEPAT (FAST ABORT)

- Attempt #1 gagal -> Breaker failure count = 1
- Attempt #2 gagal -> Breaker failure count = 2 -> State beralih ke OPEN
- Attempt #3 & #4 langsung ditolak lokal oleh Allow() tanpa call network

PROTEKSI TOTAL

- Mencegah retry membebani backend yang sedang berjuang pulih
- Waktu eksekusi caller terpangkas drastis tanpa sleep yang sia-sia
- TestIntegration_RetryWithCircuitBreaker membuktikan pemotongan ini

Circuit breakers short-circuit futile retries before they hit an already failing downstream.

Integrasi: Safe Retries Melalui Idempotency Store

Memastikan operasi non-idempoten aman dieksekusi berulang kali

SKENARIO SIMULASI

- Operasi bisnis: actualExecutions counter mencatat eksekusi nyata
- Request pertama berhasil menyimpan hasil ke Idempotency Store
- Tetapi caller mengalami transient network disconnect sebelum terima data

EKSEKUSI RETRY LOOP

- Retrier menjalankan 3 attempt pengulangan dengan request yang sama
- Store.Get(idempotencyKey) menemukan cache record transaksi sebelumnya
- Mengembalikan response tersimpan dengan label DEDUPLICATED

HASIL VERIFIKASI

- actualExecutions tetap bernilai 1 meskipun attempt berlangsung berkali-kali
- Uang nasabah hanya terdebit sekali; saldo aman dari double charging
- TestIntegration_IdempotentRetry mengonfirmasi proteksi absolut ini

Idempotency keys make retries safe by returning cached results for duplicate requests.

Verifikasi Komprehensif: 13 Test Case Lolos 100%

Pengujian menyeluruh mencakup unit, konkurensi, dan integrasi lintas modul

DISTRIBUSI TEST SUITE

- internal/circuit: 3 tests (StateTransitions, HalfOpenFailure, ZeroConfig)
- internal/deadline: 3 tests (Success, Timeout, ParentInherited)
- internal/retry: 5 tests (FirstTry, RetryUntilSuccess, ExceedMax, ContextCanceled, Bounds)
- internal/idempotency: 3 tests (GetSet, ConcurrentAccess, LazyEviction)

INTEGRATION SUITE

- TestIntegration_RetryWithCircuitBreaker -> PASS
- TestIntegration_IdempotentRetry -> PASS
- Race detector (go test -race ./...) berjalan bersih tanpa data race

DURASI & STABILITAS

- Total 13 automated test cases selesai dalam ~1.6 detik
- Semua race window diuji dengan goroutine agresif
- Zero flake, deterministik, dan reproducible

13/13 tests pass cleanly under the Go race detector, verifying concurrent correctness.

Program Demo Interaktif: cmd/demo/main.go

Demonstrasi langsung 4 skenario ketahanan dalam terminal

SKENARIO 1 & 2

- Demo 1: Context Deadline membatalkan task yang melampaui budget
- Output: Deadline propagation result: context deadline exceeded
- Demo 2: Retrier mencoba 3 kali dengan Full Jitter hingga sukses
- Output: Attempt #1, #2, #3 dieksekusi dengan jeda acak seragam

SKENARIO 3

- Demo 3: Circuit Breaker dari CLOSED -> 2 gagal -> OPEN
- Call ditolak instan saat OPEN; cooldown -> HALF_OPEN -> sukses -> CLOSED
- Output membuktikan pemulihan otomatis saat downstream sehat kembali

SKENARIO 4

- Demo 4: Idempotent Request Retry Protection
- Attempt 1: Charged $100 successfully
- Attempt 2 (Retry): Charged $100 successfully (DEDUPLICATED)

The runnable demo proves deadline cancellation, jittered retry, trip/recovery, and dedup in action.

Empat Anti-Pattern Umum Terkait Timeout

Kebiasaan buruk di industri yang memicu cascading failure di produksi

1. TIMEOUT ANGKA BESAR

- Menyetel timeout 30s-60s 'supaya aman dan tidak gampang error'
- Menyebabkan antrian thread meledak saat dependency melambat sejenak
- Harus berbasis latency P99 riil + toleransi sempit (misal 2.5× P99)

2. RETRY TANPA JITTER

- Retry terjadwal kaku pada interval eksak (1s, 2s, 4s)
- Menciptakan fenomena thundering herd yang melumpuhkan recovery backend
- Wajib acak penuh dengan Full Jitter

3. BLIND RETRY & ONLY HTTP

- Mengulang request mutasi tanpa idempotency key -> data corrupt
- Hanya memasang timeout di reverse proxy/HTTP, lupa membatasi SQL query
- PostgreSQL connection pool tetap terkuras habis oleh query gantung

Arbitrary high timeouts and jitterless retries turn minor glitches into catastrophic outages.

PostgreSQL Timeouts: Perlindungan Connection Pool

Mencegah query lambat atau lock gantung membajak seluruh koneksi database

STATEMENT TIMEOUT

- SET statement_timeout = '5s'
- Membatalkan query secara paksa jika durasi komputasi melebihi batas
- Mencegah table scan tak terindeks menghabiskan CPU database

LOCK TIMEOUT

- SET lock_timeout = '2s'
- Membatalkan transaksi jika menunggu exclusive lock terlalu lama
- Mencegah antrian DDL/UPDATE memblokir ribuan pembaca (connection pileup)

IDLE IN TRANSACTION

- SET idle_in_transaction_session_timeout = '10s'
- Memutus koneksi yang membuka BEGIN tapi tidak kunjung COMMIT
- Mencegah hold lock tak sengaja dan bloat vacuum pada tabel

Application timeouts are useless if slow database queries hold open connection pools indefinitely.

Propagasi Batas Waktu Lintas Protokol Jaringan

Meneruskan deadline melewati batas proses dan host fisik

GRPC HEADERS (STANDAR)

- Header resmi: grpc-timeout (misal 500m untuk 500 milidetik)
- Downstream gRPC client otomatis mengurangi durasi transmisi
- Menerapkan context deadline langsung ke server handler

HTTP/REST HEADERS

- Belum ada standar tunggal ISO; heterogen antar vendor
- Pola umum: Request-Timeout, X-Request-Timeout, atau Baggage
- W3C Baggage / OpenTelemetry Context Injection semakin populer

CLOCK SKEW DEFENSE

- Jangan pernah mengirim absolut wall-clock time (misal 14:00:05.123)
- Perbedaan NTP antar host (clock skew) dapat menyebabkan false timeout
- Selalu transmisikan sisa durasi relatif (remaining duration)

Transmit remaining duration, never absolute timestamps, to eliminate clock skew hazards.

Batasan Eksplisit Implementasi Lab 28

Pemisahan transparan antara simulator lab dan kebutuhan produksi nyata

IN-MEMORY IDEMPOTENCY

- Lab menyimpan key di memori dengan sync.Mutex
- Data hilang saat server crash atau restart
- Produksi butuh Redis cluster atau database transaksional

PROSES TUNGGAL (SINGLE PROCESS)

- Deadline diuji dalam context Go proses tunggal
- Tidak mencakup transmisi serialisasi wire protocol gRPC/HTTP
- Tidak memodelkan dynamic adaptive timeout berbasis P99 live

DATABASE STORAGE LAYER

- Lab membuktikan konsep melalui arsitektur software
- Konfigurasi PostgreSQL statement_timeout dibahas konseptual
- Tidak menjalankan live cluster Docker PostgreSQL

Educational implementations highlight core principles; production requires distributed state stores.

Perbandingan: Tanpa Perlindungan vs Sistem Resilien

Dampak langsung penerapan pola Timeouts & Deadlines

KONDISI TANPA PERLINDUNGAN

- Default HTTP Client Go: No Timeout (menunggu selamanya)
- Dependency lambat 60s: Thread pool terkuras, QPS anjlok ke 1.6
- Retry instan: Terjadi thundering herd retry storm
- Network timeout pada mutasi: Rentan double charge transaksi

DENGAN TIME BUDGET & RESILIENCE

- Time Budget: Eksekusi dibatasi tegas; resource segera dilepas
- Hierarchical Deadline: Downstream berhenti jika parent dibatalkan
- Full Jitter Retries: Beban tersebar merata secara statistik
- Circuit Breaker & Idempotency: Downstream dilindungi, data aman

HASIL OBSERVABILITAS

- Sistem tahan terhadap kegagalan parsial tanpa downtime total
- Resource usage (thread, socket, memory) tetap terkendali stabil
- Zero orphan goroutine / memory leak di background

Resilience patterns transform uncontrolled systemic failure into graceful degraded operation.

Prinsip Inti Perancangan Timeouts di Produksi

Tiga pilar utama untuk membangun distributed system berdaya tahan tinggi

1. SET EXPLICIT BOUNDS EVERYWHERE

- Jangan pernah menggunakan default timeout nol pada client manapun
- Pasang batas waktu pada HTTP client, gRPC, database, dan messaging queue
- Tetapkan timeout berdasarkan data observabilitas P99 latency aktual

2. ALWAYS JITTER RETRIES

- Gunakan Full Jitter untuk mendispersi waktu tunggu antar client
- Kombinasikan dengan Circuit Breaker untuk memotong retry sia-sia
- Batasi jumlah maksimal retry secara konservatif (2-3 kali)

3. SAFEGUARD MUTATIONS WITH KEYS

- Anggap timeout sebagai status indeterminate
- Kirim Idempotency-Key pada semua operasi POST / mutasi state
- Simpan status eksekusi dengan TTL untuk menghentikan double execution

Bound every call, jitter every retry, and key every mutation for absolute resilience.

SOFTWARE ENGINEERING LAB | LAB 28 RINGKASAN

Ringkasan Lab 28: Timeouts & Deadlines

KOMPONEN TERVERIFIKASI

- Context Deadline: non-blocking channel, inheritance terjaga
- Full Jitter Retrier: distribusi acak seragam, anti thundering herd
- Circuit Breaker: 3-state transisi aman konkuren (CLOSED/OPEN/HALF_OPEN)
- Idempotency Store: thread-safe dedup dengan lazy TTL eviction

HASIL VERIFIKASI

- 13 unit & integration test PASS (100%)
- Race detector clean (go test -race ./...)
- Demo interaktif berhasil membuktikan seluruh skenario

SUMBER KODE & REPOSITORI

- Repository: https://github.com/lukman-ss/software-engineering-lab/tree/main/labs/28-timeouts-and-deadlines
- Commit Hash: bdb285d35a71c9a309e83a0e4497b9dbd842ceb8
- Uji mandiri: go test -v -race ./... && go run ./cmd/demo
- Author: Lukman / lukman-ss

Distributed resilience requires time bounds at every layer: deadline, retry jitter, breaker, and dedup.